國立清華大學 電機工程學系

實作專題研究成果報告

FPGA architecture for high-frequency spot market trading systems

FPGA 架構之現貨市場高頻交易系統

專題領域：系統領域

指導教授：翁詠祿教授 (Prof. Yeong-Luh Ueng)

組員姓名：王修呈(Hsiu-Cheng Wang)、梁頊宸 (Hsu-Chen Liang)

研究期間：114 年 6 月 1 日至 114 年 11 月 24 日止，共 6 個月

Abstract

This project presents the architecture and validation plan for a Low Latency FPGA-Accelerated High-Frequency Trading (HFT) System, designed to address the inherent latency constraints of traditional CPU-based software trading systems. The fundamental goal is to utilize the parallel and pipelined processing capabilities of the FPGA to achieve sub-microsecond latency, which is critical for gaining market advantage in the rapidly evolving financial markets.

The system architecture is strictly partitioned into a Network Module and an Application Module within the FPGA, adhering to a philosophy of protocol-application separation. The Network Module implements the TCP/IP stack (including mechanisms like Delayed ACK for efficiency) and utilizes the Xilinx 10G Ethernet IP for robust physical layer connection. The Application Module encompasses the FIX 4.4 Encoder and Decoder logic, which communicates directly with the Trading Algorithm (Client FSM).

The project focuses on achieving full-stack integrity by implementing a custom FIX 4.4 protocol encoder/decoder to interface with the Taiwan Stock Exchange (TWSE) specifications. A rigorous two-phase validation plan is proposed: Network Testing uses the network_test.cpp tool (Raw Socket/pcap) to verify the network_top module's TCP/IP checksum and sequencing logic, while FIX Protocol Testing uses the Pseudo Client as a Golden Reference Model to ensure the fix_top module's encoding, decoding, and session error handling logic (e.g., Resend Request) is identical to the software implementation.

This work serves not only to validate the low-latency potential of FPGAs in real-world trading protocols but also establishes a complete, documented, and expandable architecture for future HFT system development.

摘要

本專案介紹了低延遲 FPGA 加速高頻交易 (HFT) 系統的架構和驗證方案，目的在解決傳統基於 CPU 的軟體交易系統原有的延遲限制。其根本目標是利用 FPGA 的平行和管線處理能力實現亞微秒延遲，這對於在快速發展的金融市場中贏得市場優勢非常重要。

此系統架構在 FPGA 內部嚴格劃分為網路模組和應用模組，遵循協議與應用分離的概念。網路模組實作 TCP/IP 協定堆疊（包括延遲 ACK 等機制以提高效率），並利用 Xilinx 10G 乙太網路 IP 實現穩定的實體層連線。應用模組包含 FIX 4.4 編碼器和解碼器邏輯，可直接與交易演算法（客戶端 FSM）通訊。

本專案致力於透過實現自訂 FIX 4.4 協定編碼器/解碼器來與台灣證券交易所 (TWSE) 規範對接，從而實現全端完整性。我們提出了一個嚴格的兩階段驗證計劃：網路測試使用 network_test.cpp 工具（原始套接字/pcap）來驗證 network_top 模組的 TCP/IP 校驗和及排序邏輯；FIX 協定測試使用偽客戶端作為標準參考答案，以確保 fix_top 模組的編碼、解碼和會話錯誤處理請求（例如，重新發送錯誤）。

這項工作不僅驗證了 FPGA 在實際交易協議中的低延遲潛力，還為未來的高頻交易 (HFT) 系統開發建立了一個完整、有據可查且可擴展的架構。

一、 研究動機

隨著金融市場電子化與高頻交易（HFT）的興起，速度已成為決定市場優勢的關鍵因素。傳統基於中央處理器（CPU）的軟體交易系統，由於必須依賴作業系統核心進行排程、上下文切換，以及透過 PCI Express 匯流排進行網路通訊，使得其在延遲（Latency）控制上存在難以突破的限制，難以滿足 HFT 追求的「亞微秒級」延遲需求。

本研究的動機正是為了克服這些瓶頸，將注意力轉向 FPGA（Field-Programmable Gate Array）。FPGA 憑藉其可程式化的硬體架構，允許開發者以平行化和 Pipeline 的方式設計數據處理流程，直接在硬體層實現網路協定解析、交易協定處理與策略邏輯，從而顯著縮短數據路徑，降低整體延遲。

本專題旨在：

1. 補足現有研究缺口： 建立一個涵蓋從行情解碼、訂單簿、交易策略、到送單與回報（FIX 編解碼）的完整 FPGA-based HFT 整合架構。
2. 驗證實務可行性： 透過實作符合臺灣證券交易所（TWSE）規範的 FIX 4.4 協定編解碼器，驗證 FPGA 架構在真實市場協定下維持穩定與亞微秒級低延遲的可行性。

二、 研究目的

本專題致力於建構一個完整、低延遲且可擴充的 FPGA 加速高頻交易系統，並達成以下具體目標：

1. 設計分區架構： 將系統嚴格劃分為「網路模組」（處理 TCP/IP 協定堆疊）與「應用模組」（處理 FIX 4.4 協定與交易策略），以最大化硬體平行處理的效率。
2. 硬體化核心協定：
    - 在硬體中實現 TCP/IP 協定，包括 IP 封包的編解碼、序列號管理，以及關鍵的 延遲確認 (Delayed ACK) 機制，以優化網路傳輸效率。
    - 實現符合 TWSE 規範的 FIX 4.4 訊息編碼器和解碼器，確保與真實交易所的通訊能力。
3. 建立交易執行邏輯： 實作 Trading Algorithm (Client FSM) 作為核心交易策略執行單元，能在奈秒級時間內做出買賣決策。
4. 建立嚴謹的驗證機制： 開發 C++ 軟體模擬器（Pseudo Server/Client）作為 Golden Reference Model，設計兩階段驗證流程，確保硬體模組的協定邏輯與軟體行為完全一致。

三、 理論架構

(一) TCP/IP 網路架構

本設計實作的網路會話控制模組，基於 TCP/IP 協定堆疊。TCP/IP 模型是現代網際網路的基石，它將複雜的網路通訊流程劃分為多個層次，每個層次負責特定的功能，從而實現跨異質網路的可靠數據傳輸。在數位電路（FPGA）實現中，主要聚焦於應用層、傳輸層和網際網路層的功能。

1. 網際網路協定 (Internet Protocol, IP)

   IP 協定位於網際網路層，負責無連線、不可靠的數據包（Datagram）傳輸。

   - Addressing：IP 使用 32 bits 的 IP 位址 (IPv4) 來唯一標識網路上的每個設備，確保數據包能夠被路由到正確的目的地。
   - Packet Format：IP 將上層（如 TCP）傳來的數據封裝在 IP header 中，標頭包含源地址、目的地址、總長度、存活時間 (TTL) 和 checksum 等關鍵欄位。
   - Fragmentation：IP 處理數據包在不同網路間傳輸時可能遇到的最大傳輸單元 (MTU) 限制，必要時會進行數據包的分段和重組。

2. 傳輸控制協定 (Transmission Control Protocol, TCP)

   TCP 協定位於傳輸層，提供連線導向、可靠的、按順序的位元組流服務。這是網路會話控制模組的核心實現目標。

   - Three-Way Handshake：TCP 使用 SYN (同步) 和 ACK (確認) 旗標進行三次握手，確保連線在數據傳輸開始前可靠建立。
   - Reliability：
     - Sequence Numbers：確保數據順序，檢測丟包和重複包。
     - Acknowledgement Numbers：用於通知發送端已成功接收到的下一個期望收到的序列號。
     - Retransmission：發送端在沒有收到確認（ACK）時，會根據定時器機制重傳數據。
   - Flow Control：TCP 使用 Advertised Window 機制，允許接收端通知發送端自己願意接收的數據量，防止發送端淹沒接收端的緩衝區。

3. 網際網路校驗和 (Internet Checksum)

   IP 和 TCP 標頭中均包含 16 bits 的 checksum 欄位，用於檢測數據在傳輸過程中是否發生錯誤。

   - 計算方式: 校驗和是將待保護的數據（16 bits 數據）進行 One's Complement Sum，然後對最終結果取 One's Complement Sum。
   - IP Pseudo Header: TCP checksum 的計算必須包含一個 IP pseudo header。該 pseudo header 由 IP 層的來源地址、目的地址、協議代號和 TCP 長度組成，確保 TCP 封包在最終被發送到正確的目的地且在正確的協議層被處理。

(二) FIX.4.4 交易協定

本專案的高頻交易系統以 FIX 協定 4.4 版本作為核心通訊標準，旨在與臺灣證券交易

所 (TWSE) 的 FIX 作業平台對接。FIX 協定作為國際金融交易市場的通用標準，確保了系統具備高效率、可擴展性和跨平台互通性。

1. FIX 訊息格式與結構

FIX 訊息採用 <tag>=<value> 的鍵值對格式，所有欄位之間必須使用 ASCII 碼 0x01(<SOH>) 作為分隔符號。每筆訊息的長度不固定，但結構必須遵循嚴格的三段式順序：

- Standard Header： 必須固定在訊息最前面，包含三個強制性欄位：BeginString (設為 FIX.4.4)、BodyLength 和 MsgType。
- Body： 包含實際的應用層和會話層數據，其欄位順序可不固定。
- Standard Trailer： 必須固定在訊息最後面，僅包含 CheckSum 欄位。

2. 訊息序列號與校驗和機制

(1) 序列號機制

FIX 協定使用 MsgSeqNum 確保訊息傳輸的連續性：

- 編碼原則： 訊息序號的編碼原則是每日從 1 開始。
- 連續性檢查： 接收端必須確保收到的序號是連續的。若收到的序號大於預期，則認定有訊息遺漏，必須發送 Resend Request (MsgType 2) 請求補傳。若收到的序號比預期小，則視為嚴重錯誤，必須立即停止連線。

(2) 校驗和機制 (CheckSum)

- CheckSum 算法： CheckSum 的值是將訊息中所有字元的二進制位元組值從訊息第一個欄位開始，相加至 CheckSum 欄位之前，總和除以 256 後的餘數。
- 格式要求： 最終餘數必須轉換為三個 ASCII 數字 (例如 001 到 255) 傳送，以確保在傳輸時是可見字元。

3. FIX 會話層 (Session Layer) 流程

流程階段	關鍵訊息	核心動作與時序要求
登入 (Logon)	Logon (A)	發起端 (證券商) 必須在建立通訊層連線後 60 秒內傳送 Logon 訊息。驗證成功後，交易所回覆 Logon 訊息。發起端收到交易所的 Logon 訊息後才能傳送其他訊息。
連線維護	Heartbeat (0), Test Request (1)	Heartbeat 間隔通常設為 10 秒。登入成功後，發起端可傳送 Test Request 檢查交易所回覆 Heartbeat 的序號，確保訊息沒有缺漏。
登出 (Logout)	Logout (5)	正常終止連線的方式。發起端必須收到接收端的 Logout 訊息才算完成登出。若發起端超過 5 秒未收到 Logout 回覆，將主動中斷連線。

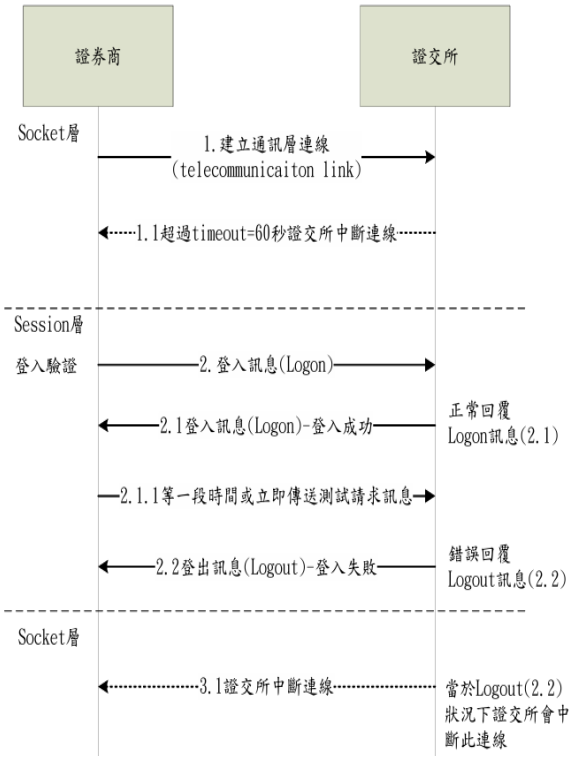


Figure 2： FIX 協定連線建立流程(圖片來源：臺灣證券交易所 FIX 4.4 電文規範 作業手冊)

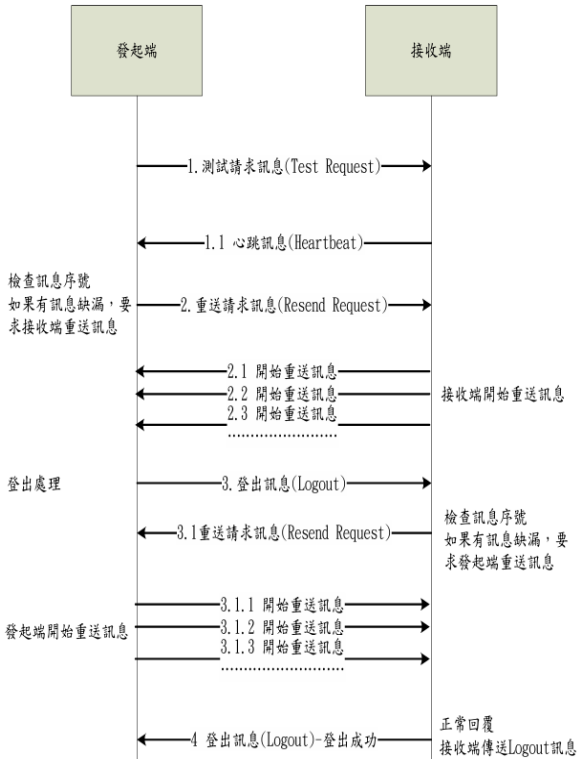


Figure 1：FIX 協定交易程序(圖片來源：臺灣證券交易所 FIX 4.4 電文規範 作業手冊)

訊息類型 (MsgType, Tag 35)	FIX 訊息名稱	應用層次
A	Logon (登入)	會話層
0	Heartbeat (心跳)	會話層
2	Resend Request	會話層
5	Logout (登出)	會話層
D	New Order Single (新訂單)	應用層
F	Order Cancel Request	應用層
G	Order Cancel/Replace	應用層
H	Order Status Request	應用層

四、 研究方法

(一) 整體架構

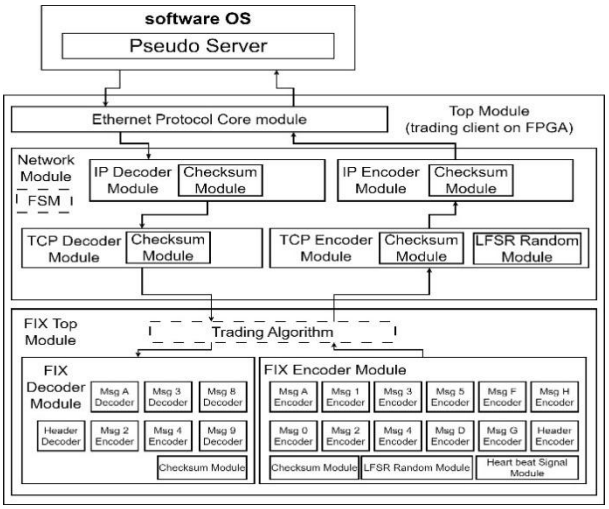


Figure 3：交易系統整體架構圖

本高頻交易系統將功能劃分為軟體層（Software OS）和硬體層（Client on FPGA），並在硬體層內進一步區分為網路模組 (Network Module) 和應用模組 (Application Module)。這種分區設計旨在最大化 FPGA 的低延遲優勢，並確保協定處理的穩定性和可擴展性。

- 軟體層 (Software OS) / Pseudo Server：
  - 職責： 提供外部連線的 Golden reference model 和測試環境。
  - 邏輯： 部署 Pseudo Server 模擬交易所或券商的行為，用於在開發階段驗證 FPGA 的通訊和協定邏輯。
- 硬體層 (Client on FPGA)：

FPGA 內部邏輯被清晰地劃分為兩個核心模組，實現了協定與應用分離。

(1) 網路模組 network_top

此模組專注於處理標準協定和物理層連接，以確保數據的快速、可靠傳輸。

子元件	設計功能與目的
Ethernet IP	處理乙太網路的物理層 (PHY) 連接和數據鏈路層。採用現成的 IP 核心（如 Xilinx 1G/10G Ethernet IP），是為了確保與實體網路的連接穩定性，並避免在複雜的 PHY 層出現錯誤。
Network layer & Transport Layer	實現 TCP/IP（或 UDP/IP）協定棧的硬體邏輯。這包括 IP 封包的編解碼、序列號管理、以及 TCP 三次握手和延遲 ACK 等邏輯。此模組的正確性是低延遲通訊的基礎。

1. 應用模組：fix_top

此模組專注於處理交易協定和核心交易邏輯，是決定交易速度和策略準確性的關鍵。

子元件	設計功能與目的
FIX 4.4 Decoder	解碼接收到的原始 FIX 訊息。將字串形式的 FIX 數據（來自網路層）解析成結構化的、易於硬體處理的交易參數，並將結果寫入

	Order In Register。
FIX 4.4 Encoder	編碼待發送的交易指令。將 Trading Algorithm 產生的結構化指令（來自 Order Out Register）轉換為標準的 FIX 4.4 格式字串（Payload），供網路模組封裝。
Order In / Out Register	作為協定處理層與交易邏輯層之間解耦的介面。Order In Register 暫存解析後的行情數據，Order Out Register 暫存演算法生成的委託指令。
Trading Algorithm (Client FSM)	核心交易策略執行單元。 根據 Order In Register 的行情數據和內建的 FSM 邏輯，在納秒級時間內做出買賣決策，並將結果輸出到 Order Out Register。

(二) 網路架構



Figure 4：network 模組 TCP 協定有限狀態機

TCP 會話控制模組 (Network Top)

1. 模組概述與目的

Network Top 模組是實現 TCP/IP 協定堆疊核心會話控制邏輯的有限狀態機（FSM）。其主要目標是高效且可靠地管理網路連線的建立、數據傳輸和 TCP 封

包的確認機制，特別是實現了 Delayed Acknowledgement 機制，以優化網路頻寬的使用。本 FSM 包含三個主要狀態：IDLE (閒置/等待初始連線狀態)、LOGON（連線發起與同步狀態）和 CONNECT（數據傳輸狀態）。

2. 狀態機核心流程與邏輯分析

該 FSM 遵循標準的 TCP 連線管理，但邏輯經過簡化和優化以適應硬體實現。

(1) IDLE 狀態 (閒置與初始化)

起始狀態 ，等待外部啟用訊號 (enable) 或接收到有效的 TCP 解碼數據 (tcp_decode_valid) 以啟動連線 。

(2) LOGON 狀態 (連線發起與同步)

此狀態專門處理 TCP 連線建立過程中的三次握手 (Three-Way Handshake) 流程

(3) CONNECT 狀態 (數據傳輸與延遲確認)

此狀態是數據交換的主體，負責處理數據發送 (PUSH) 以及高效的 ACK 確認邏輯，其設計核心為 tx_enable 訊號。

3. 設計優化：Delayed ACK

本模組的核心優化在於 CONNECT 狀態下的延遲 ACK 邏輯。透過使用一個大型計數器 (delay_ack_counter)，系統避免了在接收到每個數據包時立即回覆 ACK，而是選擇將 ACK 延遲至：

(1) 本地(FPGA)有數據 (tx_enable = 1) 需要發送時，將 ACK 附在數據中。

(2) 延遲計數器達到 1000000 個時脈週期上限時，且有未確認的 ACK 需求，則強制發送獨立的 ACK 封包。

這種機制有效地減少了網路上的 ACK 流量，提升了整體傳輸效率。

(三) Ethernet 架構

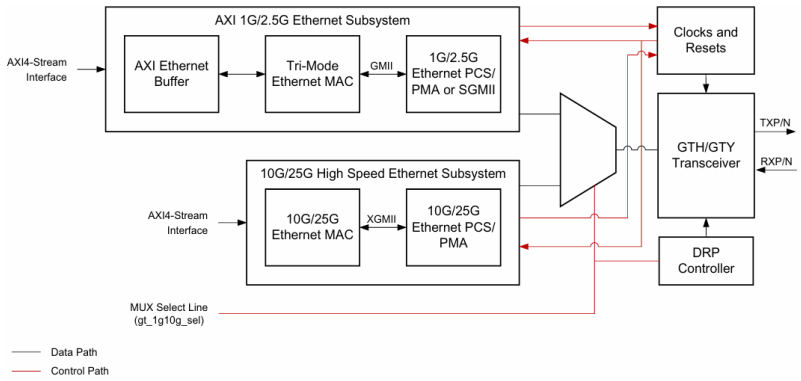


Figure 5：Ethernet Core Block Diagram

本專案的高頻交易硬體架構將採用 Xilinx UltraScale 架構中的 1G 到 10G 乙太網路動態切換解決方案 (XAPP1307)。

1. 模組架構與功能

該解決方案將 AXI 1G/2.5G 乙太網路子系統 與 10G/25G 高速乙太網路子系統整合，並利用一個共用的 GTH/GTY 收發器 (Transceiver) 核心處理物理層序列化。

組件	類型	功能與應用
GTH/GTY Transceiver	物理介面 (PHY)	執行序列化與解序列化功能，是 1G 和 10G 協定共用的硬體資源[7]。
DRP Controller	控制核心 (FSM)	透過動態重配置埠 (DRP) 介面，依據用戶輸入的選擇線 (gt_1g10g_sel) 讀寫 GT 暫存器，實現速率的動態切換[8]。

2. 專案修改方向與精簡策略

- IP Core 的 AXI4-Stream 介面是 MAC 層與上層之間的標準用戶介面。
- 必須調整 AXI4-Stream 的數據寬度，使其能夠傳輸完整的 User Payload，該 Payload 包含 FIX 數據、FIX Header、TCP Header 和 IP Header 等所有上層協定資訊。
- 基於本專案主要需求為 10Gb/s 高速交易，將精簡電路設計，僅使用和致能 10G/25G 乙太網路子系統。
- 移除或停用 1G/2.5G 的相關邏輯和數據路徑，以減少不必要的邏輯資源消耗，並簡化控制邏輯。
- 設計將使用單一 156.25 MHz 參考時脈，並透過 DRP 配置 GT，使其鎖定於 10Gb/s 速率，採用 10GBASE-R 協定。

(四) IP protocol encoder

本模組是網路發送路徑中的一個組合邏輯區塊，用於將上層（TCP 層）提供的數據封裝成一個標準的 IPv4 封包，以準備傳輸 。

1. 功能概述

該邏輯接收來自 TCP 模組的完整 TCP 封包數據 (rx_tcp_data) ，並根據預定義的常量 和內部序列號 (ident) ，即時建構一個完整的 IP 封包暫存器 (tx_ip_reg) 。它還負責更新 IP 封包識別碼 (ident) 的序列號，以便發送下一個封包。

2. IP 封包標頭建構邏輯 (tx_ip_reg)

IP 封包標頭的建構遵循 IPv4 協定標準，其位元分配如下：

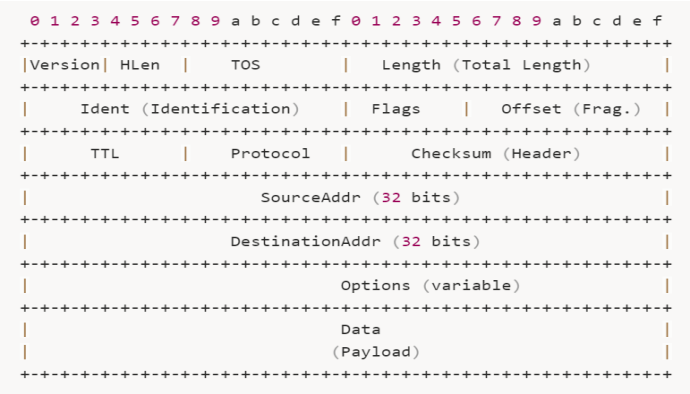


Figure 6：IP header 資料格式

3. 數據封裝與校驗和處理

- 上層數據封裝： 來自 TCP 模組的數據 (rx_tcp_data) 被放置在 IP 標頭之後，即 tx_ip_reg 的最低位元區段 [(PAYLOAD_LEN + TCPH_LEN)*8-1:0] 。

- Checksum 計算： 外部的 checksum 模組負責計算 IP 標頭的 checksum。計算時，輸入數據 (tx_ip_reg) 的 checksum 欄位 (位元 95:80) 會被暫時設為 16'd0，然後計算出的結果 (data_checksum) 再寫入 tx_ip_reg 的對應位置。

(五) IP protocol decoder

本模組是網路接收路徑中的一個組合邏輯區塊，用於對接收到的 IPv4 封包進行驗證和解封裝，並將上層數據提取給 TCP 層

1. 功能概述

該邏輯接收包含 IP 標頭和 TCP 數據的完整 IP 封包 (rx_ip_data) ，並對其進行多重檢查。只有當所有檢查通過時，才會拉高 ip_decode_valid 訊號並輸出提取的 TCP 數據 (tx_tcp_data)。

2. IP 封包解碼與驗證邏輯

解碼器透過組合邏輯，對接收到的 IP 封包 (rx_ip_data) 進行嚴格的欄位驗證 ：

- 欄位匹配： 驗證封包的版本 (VERSION =4'd4), 標頭長度 (HEADER_LEN = 4'd5), 服務類型 (TOS = 8'd0), 協議 (PROTOCOL = 8'd64), 來源地址 (SRCADDR) 和目的地址 (DESADDR) 均須符合預期參數。
- 識別碼檢查： 接收到的識別碼 (ident) 必須大於前一個接收到的識別碼 (last_ident)，以確保數據包的順序性。
- 校驗和檢查： 透過 checksum 模組 ，計算接收數據 IP header 的 checksum，並與封包中攜帶的校驗和欄位進行比較。

3. 數據提取與輸出

- 解碼有效性：僅當上述所有驗證條件（包括版本、長度、地址、協議、識別碼順序以及 checksum）全部滿足時 ，ip_decode_valid 才會被拉高 ，指示上層模組數據已準備好。
- 數據提取： 若驗證成功，TCP 數據部分 (rx_ip_data[(PAYLOAD_LEN + TCPH_LEN)*8-1:0]) 被提取並暫存於 tx_tcp_reg ，最終輸出至 tx_tcp_data 供 TCP 層使用 。
- 識別碼更新： 在每個時脈週期，當前接收到的有效識別碼 (ident) 會更新到 last_ident 暫存器中 ，用於下一個封包的順序檢查 。

(六) TCP 架構

本章節描述了網路堆疊中 TCP 編碼器 (tcp_encoder) 和 TCP 解碼器 (tcp_decoder) 模組，它們共同負責 TCP 封包的生成、傳輸數據的提取以及驗證。

1. TCP 編碼器

本模組是 TCP 發送路徑中的核心組合邏輯區塊 。其核心功能是根據狀態機提供的 flag，即時建構和格式化待傳輸的 TCP 封包 header 和 pseudo IP header 。它接收應用層數據 (rx_fix_data) 並將其封裝為完整的 TCP 數據包 (tx_tcp_data) 。

(1) 同步 (SYN) 封包生成

當 flag 設為 SYN 時，模組執行連線建立流程（三次握手的第一步）所需的初始化動作：

- 序列號初始化： 序列號 (seq_num) 欄位被賦予由 rand_val 組合成的初始序列號 (ISN)，以確保連線的唯一性。
- 確認號歸零： 確認號 (ack) 欄位被設為 0。
- 端口設定： 來源 port 代號為 rand_val 加一個固定偏移量 (16'd1024)，目標 port 代號為預定義常量 DSTPORT。
- advertise window： adv_window 根據前一狀態的視窗值 last_adv_window 進行調整，若前值非零則歸零，否則設為固定值 302。
- header 長度與 flag： HDRLEN 和 SYN flag 被寫入指定位置。
- checksum： TCP checksum 欄位暫設為 0，等待後續的 checksum 計算模組填入。

(2) 常規 (非 SYN) 封包生成

當 flag 不為 SYN 時（例如 ACK、PUSH 或 PUSH|ACK），模組處理數據傳輸和確認流程：

- 序列號和確認號遞增： 序列號 (seq_num) 和 確認號 (ack) 均採用上一次發送的值 (last_seq_num, last_ack) 遞增 1，以確保數據流的連續性。
- 其他欄位： port 代號、advertise window 和 flag 等其餘欄位的賦值方式與 SYN 封包邏輯保持一致。

2. TCP 解碼器

本模組是 TCP 接收路徑中的解封裝邏輯，負責對接收到的 TCP 封包進行驗證 。它檢查封包的 checksum 和與 flag，並在驗證成功時，提取應用層數據 (tx_fix_data) 並啟動發送訊號 (tx_enable)。

- Pseudo header 計算： 與編碼器類似，首先根據參數計算一個 pseudo header，並與接收到的數據一同用於 checksum 計算 (checksum_in)
- 數據有效性判斷： 接收到的數據包需通過三項檢查才能認定為有效：

(1) IP 解碼有效性： 外部輸入 ip_decode_valid 必須為真 。

(2) PUSH flag 檢查： 接收封包的 flag 欄位必須設置 PUSH flag 。

(3) checksum 檢查： 接收封包攜帶的 TCP checksum 必須與內部計算的 data_checksum 相匹配。

- 數據提取與控制輸出：

(1) 若所有條件成立，內部發送啟動訊號 (tx_enable_reg) ，同時應用層從封包中提取數據 (tx_fix_reg)。

(2) tx_enable 輸出信號在時脈上升緣被拉高，表示已成功解碼並提取數據 。

(3) 同時記錄了接收到的 source port (rcv_src_port) 和 destination port (rcv_dst_port) 。

(七) checksum 電路

1. 模組目標與功能

checksum 模組是一個純組合邏輯電路，專門用於計算 IP header 和 TCP header 所需的 16 bits Internet Checksum。該模組遵循 RFC 標準，將輸入的任意長度數據 (data)

視為一系列 16 bits 數字，然後計算這些字的 One's Complement Sum，最終輸出 16 bits 的 checksum 結果 (data_checksum)。

2. checksum 計算邏輯

該模組透過參數 SIZE 定義輸入數據 (data) 的總位元數，並透過組合邏輯即時執行 checksum 的計算：

(1) 分組定義 (PACK): 模組首先計算輸入數據中包含 16 bits 字的數量。

(2) 16 bits 字累加: 邏輯使用一個迴圈 (for (i = 0; i < SIZE-16; i = i + 16)) 將輸入數據從低位開始，以 16 bits 為單位進行累加。累加的結果儲存在 16 bits 的陣列 sum 中，其中 sum[i>>4+1] 儲存了當前累加的和。

計算過程實質上是將所有的 16 bits 數據進行 16 bits 的一補數加法。由於累加過程中可能會有溢位，這些溢位（Carry）會自動保留在 sum[i]的高位。

(3) 處理溢位 (Carry End-Around Add): 最終累加結果位於 sum[PACK-1]。為了完成 one's complement sum，必須將 16 bits 以外的高位（溢位）加回到低 16 bits 上。

- 溢位處理公式：

  將 sum[PACK-1]的高 16 bits（溢位）與低 16 bits（和）相加，即：

$$\text{Sum} = (\text{sum}[\text{PACK} - 1] \gg 16) + (\text{sum}[\text{PACK} - 1] \& 16'h\text{ffff})$$

(4) 最終 checksum： data_checksum 是對上述溢位處理後的 16 bits 結果取 One's Complement，即：

$$\text{data}_{\text{checksum}} = \sim(\text{Sum}) \& 16'h\text{ffff}$$

(八) fake random 電路

本模組旨在實現一個 Pseudo-Random Number Generator（PRNG），採用 Linear Feedback Shift Register（LFSR）架構。LFSR 是一種高效且資源佔用低的時序電路。

1. 電路結構與原理

參數/信號	位元寬度	功能描述
rnd	16 位元	輸出當前的偽隨機數序列，同時作為移位暫存器的核心狀態。
feedback	1 位元	異或回饋值，決定下一個狀態的最低位。
taps	N/A	[16], [14], [13], [11] 位元（對應 Verilog 中的 rnd[15], rnd[13], rnd[12], rnd[10]）。

2. 運作邏輯

(1) Feedback Calculation：

- 電路採用斐波那契型 LFSR 結構，從暫存器狀態中抽取特定位元（Taps）進行異或運算。
- 選擇的 Tap 位元是 16, 14, 13, 11（從 1 開始數）。
- 回饋信號 feedback 的組合邏輯表達式為：

$$\boldsymbol{feedback = rnd[15] \oplus rnd[13] \oplus rnd[12] \oplus rnd[10]}$$

(2) 移位與時序控制：

該電路是一個同步時序邏輯。在每個時脈上升沿 (posedge clk)：

- 暫存器 rnd 的當前狀態向右（低位）移動一位（rnd[15] 移至 rnd[14], … , rnd[1]

移至 rnd[0]）。

- 新的回饋位元 (feedback) 被寫入到最高位 rnd[15]。

$$\boldsymbol{rndnext} <= \{\boldsymbol{rnd}[\mathbf{14}:\mathbf{0}], \boldsymbol{feedback}\}$$

(3) 初始化與種子 (Seeding)：

- 異步重置 (negedge rst_n) 時，暫存器被初始化為種子值 16'h1。
- 注意： LFSR 的種子值必須是非零的，以避免陷入全零狀態，從而停止序列生成。

(九) FIX 架構

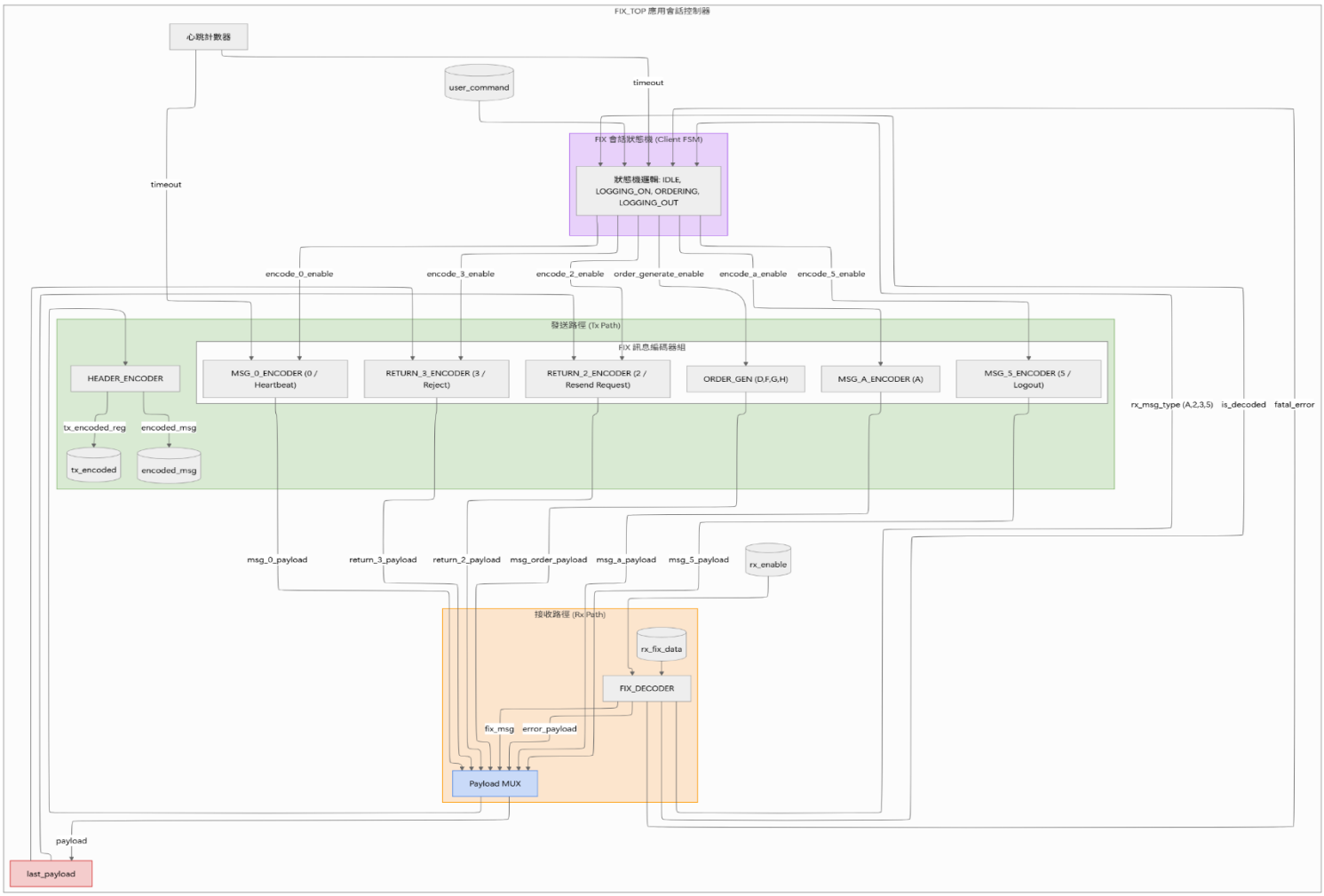


Figure 7：FIX 協定交易處裡模組架構

1. 狀態機結構與類型

- 與狀態相關： 大多數的輸出致能訊號是與當前狀態直接關聯的。例如，在進入 LOGGING_ON 狀態時，encode_a_enable 預設開啟，以發送登入請求（FIX Message Type A）。
- 與輸入相關： 在主要的交易狀態 ORDERING 中，輸出（如 encode_2_enable, encode_3_enable, encode_5_enable）直接受到即時輸入條件（如接收到的訊息類型 rx_msg_type 為 "2" 或 "3"）的影響，用於即時處理交易回覆或錯誤。

2. 核心會話流程分析

該 FSM 模擬了一個標準的 FIX 會話流程，確保協議執行的嚴格性：

(1) IDLE： 作為系統的初始和最終狀態。所有輸出致能訊號均被清零。狀態機僅在接收到外部 user_command == "START" 命令時，才會轉入連線發起狀態 LOGGING_ON。

(2) LOGGING_ON： 此狀態負責建立會話層連線。

- 發起登入： 預設輸出 encode_a_enable = 1，用於持續發送登入請求 (MsgType=A)。
- 保持與成功： 若訊息尚未解碼（!is_decoded），則保持在該狀態。只有當成功接收到已解碼且類型為 A 的登入成功回覆時，才會轉入 ORDERING 狀態。
- 失敗處理： 任何不符合預期（即登入失敗）的輸入，都會導致系統轉入 LOGGING_OUT 狀態。

(3) ORDERING： 此為主要的業務邏輯狀態。

- 發送訂單： 預設輸出 order_generate_enable = 1，用於發送訂單訊息。
- 回覆處理： 系統持續監測接收訊息。收到類型 "2" 或 "3" 的訊息時，致能對應的 encode 訊號以處理訂單回覆或交易狀態更新，並保持在 ORDERING 狀態。
- 致命錯誤： 若在交易過程中觸發 fatal_error 旗標，系統會立即致能 encode_5_enable（用於發送登出通知 MsgType=5）並強制轉入 LOGGING_OUT 狀態。

(4) LOGGING_OUT： 專門用於安全地結束會話。

- 發送登出： 預設輸出 encode_5_enable = 1，用於發送登出訊息 (MsgType=5) 或錯誤確認。
- 完成與保持： 系統保持在此狀態，直到收到已解碼且類型為 5 的登出確認訊息，之後才會安全轉移回 IDLE 狀態，等待下一次連線。

(十) FIX Encoder 與 Decoder 電路設計方向：

1. FIX 訊息解析器 (Decoders & Utility)

函數名稱	功能類型	關鍵解析目標	核心職責
SearchMsgType	核心解碼與驗證	所有強制標籤 (8, 9, 35, 34, 49, 56, 52)	會話完整性檢查：執行 CheckSum, BodyLength, Tag 順序和序列號跳號檢查。若發現跳號，則返回 2 (Resend Request)。
DecodeMessage	通用 Tag 查詢	任意 target_tag	基礎解析函數，從 FIX 字串中查找並返回指定 Tag 的 Value。
ReceiveA	Logon 專用解析	Logon 訊息 (MsgType=A)	解析登入訊息中的所有關鍵標籤 (如 98, 108, 95, 96) 到結構體，並驗證 MsgType 是否為 A。
ReceiveOrder	應用層接收處理	訂單相關訊息	判斷接收到的訊息是否為訂單相關類型 (New Order, Cancel Request, 等)。

getChecksumFromFIX	工具函式	Tag 10 的值	從接收到的 FIX 訊息結尾提取出校驗和 (CheckSum) 值。

五、 驗證方法與研究成果

為了確保最終的 Verilog 電路設計能夠正確遵循 FIX 協定和 TCP/IP 網路規範，本專案開發了兩套軟體模擬器：Pseudo Server 和 Pseudo Client，以及作為驗證基礎的網路層測試邏輯。

(一) Pseudo Server 模組：最終系統連線測試環境

Pseudo Server (實作為 TaifexOrderServer.cpp) 模組的角色是模擬交易所或券商伺服器端的通訊行為。

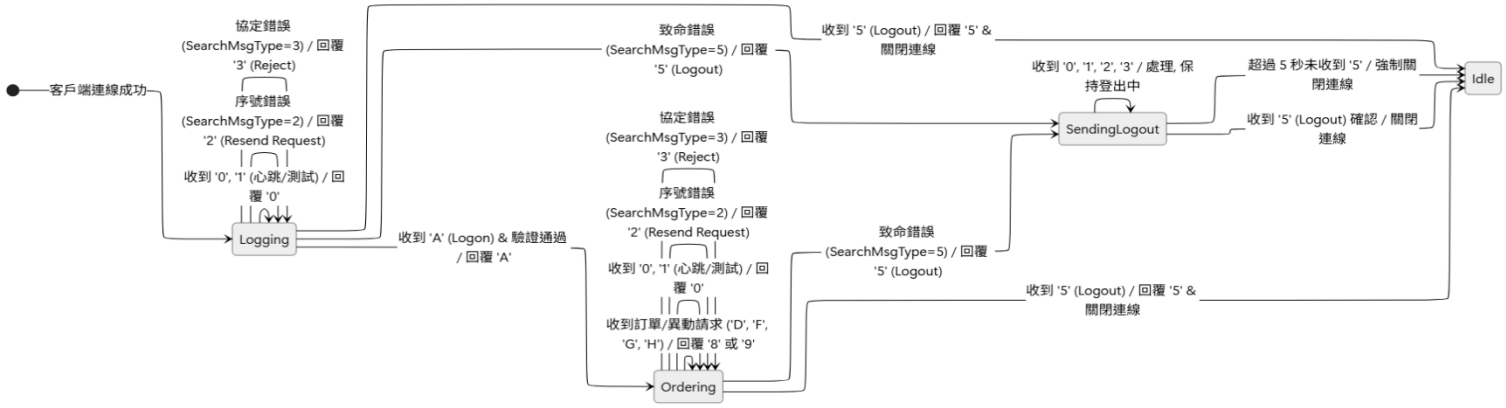


Figure 8：Pseudo Server 程式(虛擬交易平台)交易處理流程圖

- 基礎架構： 採用標準 C++ Socket 程式設計 (bind, listen, accept) 部署於軟體作業系統。
- 多線程處理： 程式為每個建立的連線（Session）建立獨立的線程，以實現並行處理多個 FIX 會話。
- FIX 會話邏輯： 實現了完整的 FIX Session Layer 狀態機 (State::Logging、State::Ordering、State::SendingLogout)，能夠正確處理登入 (MsgType=A)、心跳 (MsgType=0)、重發請求 (MsgType=2) 等會話訊息。
- 交易回覆：使用 ifstream 函式庫接收 20 筆對話資料進行測試，每次訊息來回最多花費 2 milliseconds，可以提供極高速的交易訊息對話。

```
ted@docker-desktop:/home/HFST/PseudoServer/build$ ./TaifexOrderServer
Server listening on 0:9000 with 1 sessions.
Waiting for connection...
Connection accepted.
[Session 0] Connected from 192.168.65.3
buffer size: 101
tagNumber: 1, tag: 8, value: FIX.4.4
tagNumber: 2, tag: 9, value: 79
tagNumber: 3, tag: 35, value: A
tagNumber: 4, tag: 34, value: 1
tagNumber: 5, tag: 49, value: T116001
tagNumber: 6, tag: 56, value: XTAI
tagNumber: 7, tag: 52, value: 01251123-15:31:45.689
[SERVER] port 9000, session 0, send A
Ordering
buffer size: 229
tagNumber: 1, tag: 8, value: FIX.4.4
tagNumber: 2, tag: 9, value: 206
tagNumber: 3, tag: 35, value: D
tagNumber: 4, tag: 34, value: 2
tagNumber: 5, tag: 49, value: T116001
tagNumber: 6, tag: 56, value: XTAI
tagNumber: 7, tag: 52, value: 01251123-15:31:45.692
[SERVER] port 9000, session 0, send 8
```

```
ted@docker-desktop:/home/HFST/PseudoClient/build$ ./TaifexOrderClient
Enter the IP of Server: 192.168.65.3
Connected to server.
tagNumber: 1, tag: 8, value: FIX.4.4
tagNumber: 2, tag: 9, value: 79
tagNumber: 3, tag: 35, value: A
tagNumber: 4, tag: 34, value: 1
tagNumber: 5, tag: 49, value: XTAI
tagNumber: 6, tag: 56, value: T116001
tagNumber: 7, tag: 52, value: 01251123-15:31:45.691
Ordering
Input Order MsgType: NewOrder
Ordering
tagNumber: 1, tag: 8, value: FIX.4.4
tagNumber: 2, tag: 9, value: 273
tagNumber: 3, tag: 35, value: 8
tagNumber: 4, tag: 34, value: 2
tagNumber: 5, tag: 49, value: XTAI
tagNumber: 6, tag: 56, value: T116001
tagNumber: 7, tag: 52, value: 01251123-15:31:45.692
Ordering
Input Order MsgType: NewOrder
Ordering
tagNumber: 1, tag: 8, value: FIX.4.4
```

Figure 9：Pseudo Server(虛擬交易平台) (左)、Pseudo Client(軟體版本客戶端) (右)

- Pseudo Server 接收連線(A)、回傳連線確認(A)、接收交易(D)、回傳結果(8)
- Pseudo Client 直接傳送連線(A)、接收連線確認(A)、發送交易(D)、接收接果(8)

(二) 階段性模組驗證

本專案的短期測試規劃將分為兩個主要階段進行，旨在利用現有的 C++ 軟體程式碼作為 Golden Reference Model，針對不同的 Verilog 硬體模組進行階段性驗證，確保功能和協定相容性。

短期測試計畫：階段性模組驗證

1. 網路層驗證 (Network Testing)

此階段的目標是驗證 network_top 模組的網路封包編解碼功能，確保其能夠正確處理標準的 TCP/IP 封包結構、控制旗標以及校驗和。

使用工具：network_test.cpp 程式碼（基於 pcap 和 Raw Socket）。

驗證流程：

(1) 封包編碼測試（給予電路測試）：

- 使用 network_test.cpp 中的 ip_encoder、tcp_encoder 和 ether_sender 函數，在 C++ 層面精確構造出測試封包（例如 TCP SYN, ACK, PUSH 等各種組合），作為 network_top 模組的輸入測試向量。

(2) 封包解碼測試（驗證電路輸出）：

- 在 network_top 模組生成輸出封包後，使用 network_test.cpp 的 pcap 捕捉功能和 packet_handler 函數來解析硬體輸出的封包。

驗證結果：

(1) decoder 與 decoder 能夠通過 100 筆 ip 封包與 100 筆 payload 的功能驗證，具有 3-way handshake，delayed ACK 與簡易 congestion control 能力。

(2) 每個時脈週期皆可傳送一筆封包數據，傳送到解碼總共耗時 2 個週期。

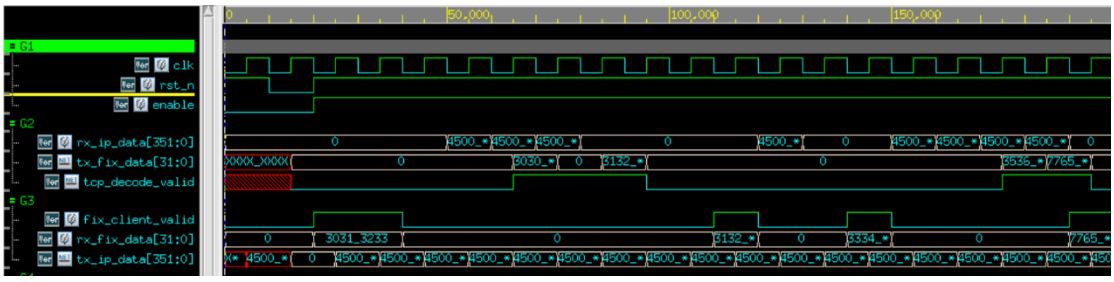


Figure 10：使用 nWave 顯示 network module 模擬結果

2. FIX 協定驗證 (FIX Protocol Testing)

此階段的目標是驗證 fix_top 模組的 FIX 應用層邏輯和會話層狀態機，確保其對 FIX 訊息的編解碼和錯誤處理與軟體一致。

使用工具：Pseudo Client 程式碼（主要利用 ClientMessenger 類別中的編解碼函數）。

驗證流程：

(1) 訊息編碼驗證：

- 將相同的輸入數據（例如，用於下單的 Order Request 參數）分別輸入到 Pseudo Client 的 ClientMessenger::MakeXxx 函數和 fix_top Verilog 模組中。

- 驗證內容： 比對軟體輸出的 FIX 格式字串（Payload）與硬體生成的字節流是否完全相同，包括 BodyLength 和 CheckSum 的計算結果。

(2) 訊息解碼與錯誤邏輯驗證：

- 構造特定的錯誤訊息（例如，序列號錯誤或登入失敗訊息）作為輸入。
- 將此輸入分別給 Pseudo Client 的 ClientMessenger::SearchMsgType 函數和 fix_top 模組。
- 驗證內容： 確認 fix_top 模組的輸出（例如，回應的狀態、是否觸發 Resend Request 邏輯、是否返回 RejectSession）與 C++ 軟體程式的回傳結果（return 0/2/3/5）完全一致。

(三) 電路合成

採用 Synopsys Design Compiler 進行 synthesis，使用了 slow_vdd1v2 standard cell library

1. Synthesis 設定

- 設計與環境：在 PVT_1P08V_125C 條件下操作。所有輸入被模擬為由 DFFHQX1 驅動，所有輸出被驅動到 DFFHQX1 的 D 端。
- 時序約束： 設置了單一時脈 clk（週期為 6.0ns），並將其視為理想網路 (set_ideal_network)。輸入輸出延遲均為 0.6ns。
- 優化策略： 使用了 compile_ultra 進行全功能 synthesis，並通過-gate_clock 選項啟用 Clock Gating 優化，且時脈門控單元的最大 fanout 被設置為 10。同時，設置了 set_max_fanout 20.0 作為 DRC rule。

2. Synthesis 結果分析

時序裕量計算

$$Slack = Data\ Required\ Time\ -\ Data\ Arrival\ Time$$

$$Slack = 3.0000\,ns\ -\ 1.3101\,ns\ =\ 1.6899\,ns\ (MET)$$

Module name	Area	Ratio	Power	Ratio	Observation
TCP_ENCODER_0	11348.5863	31.2%	0.507 mW	32.2%	Area and Power Hotspot
IP_DECODER_0	9341.7302	25.7%	0.387 mW	24.6%	Second Largest Consumption Module
IP_ENCODER_0	7586.2442	20.9%	0.363 mW	23.1%	
TCP_DECODER_0	7548.9662	20.8%	0.303 mW	19.3%	

Table 1：依模組分析面積與功耗比例

Area Type	Result	Ratio	Observation
Non-combinational	31674.330707	87.1%	Occupies the main area; design is register-intensive.
Combinational	4689.504105	12.9%	

Table 2：面積表現

Power Type	Result	Percentage of Total Power (%)
Leakage Power	0.927 mW	37.08%
Internal Power	1.552 mW	62.08%
Switch Power	0.0209 mW	0.84%

Table 3：各類功耗類型比例

六、 總結

綜合結果表明設計在功能上符合時序要求，具有充足的裕量。然而 TCP_ENCODER_0 模組在面積和功耗上均為設計瓶頸。未來的優化工作應集中於精簡此模組的狀態機或數據暫存結構，並考慮採用更低洩漏的單元庫以進一步降低總功耗。

本專題成功地設計並驗證了一個基於 FPGA 加速的低延遲高頻交易系統的完整架構。針對傳統 CPU 系統因作業系統核心排程、PCIe 傳輸以及多層次網路協定堆疊所導致的延遲瓶頸，本系統採用了硬體加速方案，將所有對延遲敏感的邏輯直接部署於 FPGA 上。

關鍵設計成果包括：

(一) 分區架構：系統被明確劃分為網路模組（處理 TCP/IP 和實體連接）與應用模組（處理 FIX 協定編解碼與交易決策），確保了協定處理的穩定性與策略執行的即時性。

(二) 協定硬體化：實現了符合 TWSE 規範的 FIX 4.4 編解碼器、具備 Delayed ACK 機制的 TCP 會話控制器，以及能夠在單一週期內計算的網際網路校驗和電路。

(三) 嚴謹的驗證策略：透過將 C++ 程式碼 (Pseudo Server/Client) 作為 Golden Reference Model，設計了兩階段驗證：

- Network Testing：利用 network_test.cpp 驗證 network_top 模組的 TCP/IP 封包編解碼和 Checksum 邏輯。
- FIX Protocol Testing：利用 Pseudo Client 驗證 fix_top 模組的 FIX 訊息編碼、解碼和序列號錯誤處理邏輯與軟體行為一致。

根據前面說的內容，本專題不僅補足了現有研究中關於完整 FIX 協定硬體架構的缺口，更提供了一個可擴充的實作範例，驗證了 FPGA 在高頻交易市場中的實務可行性和競爭優勢。

七、 參考資料

1. Taiwan Stock Exchange (TWSE), “FIX4.4 電文規範作業手冊” , Taiwan Stock Exchange, Taipei, Taiwan, *Online*. Available: https://www.twse.com.tw/staticFiles/product/broker/b002000176.doc. Accessed: Oct. 13, 2025.
2. H.-A. Chen, “A Low Latency FPGA-Accelerated High-Frequency Trading System,” M.S. Thesis, Dept. of Electrical Engineering, National Tsing Hua Univ., Hsinchu, Taiwan, 2021.
3. Q. Yi, M. Shi, and G. Zhong, “Design and FPGA implementation of ten gigabit Ethernet MAC controller,” in *2017 IEEE 2nd Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, Chongqing, China, Mar. 2017
4. C. Leber, B. Geib, and H. Litz, “High Frequency Trading Acceleration Using FPGAs,” in *2011 21st International Conference on Field Programmable Logic and Applications*, Chania, Greece, Sept. 2011.